

Description

Accepted

Editorial

Solutions

Submissions

← All Submissions

🔗

Accepted

20 / 20 testcases passed

Editorial

Solution

👤 yousab21

submitted at Mar 01, 2026 19:40

🕒 Runtime

0 ms | Beats 100.00%

Analyze Complexity

💾 Memory

9.64 MB | Beats 8.96%

Category	Value
Runtime	0 ms
Memory	9.64 MB

Code

C++

```
1 //the way i made 2 queues represnt a stack is
2 // at any time there will be an active queue and an empty queue
3 //the active queue front will always match the stack top (ill explain h
4 //so when popping or returning top the active queue works excacly as a s
5
6 #include<queue>
7 class MyStack {
8 public:
```

View more

More challenges

232. Implement Queue using Stacks

Write your notes here

Code

C++

Auto

🔗

📄

🔄

🔍

```
2 //the way i made 2 queues represnt a stack is
1 // at any time there will be an active queue and an empty queue
3 //the active queue front will always match the stack top (ill explain how i do that in the push method)
1 //so when popping or returning top the active queue works excacly as a stack
2
3 #include<queue>
4 class MyStack {
5 public:
6     queue<int> q1;
7     queue<int> q2;
8     int active;
9     MyStack() {
10         active = 1;
11     }
12
13     void push(int x) {
14         //when you want to push an element to the stack
15         //we push it into the non-active queue so that the element that would be
16         //the top of the stackc is now the front of the non-active queue
17         //then we copy the other elements in the active queue
18         //to tge other queue and make that the active one
19         //and we repeat every time we pusha an element
20         if(active == 1){
21             q2.push(x);
22             while(!q1.empty()){
23                 q2.push(q1.front());
24                 q1.pop();
25             }
26             active = 2;
27         }
28         else if(active == 2){
29             q1.push(x);
30             while(!q2.empty()){
31                 q1.push(q2.front());
32                 q2.pop();
33             }
34             active = 1;
35         }
36     }
37
38     int pop() {
39         int stackTop;
40         if(active == 1){
41             stackTop = q1.front();
```

Saved

Ln 3, Col 40

Testcase

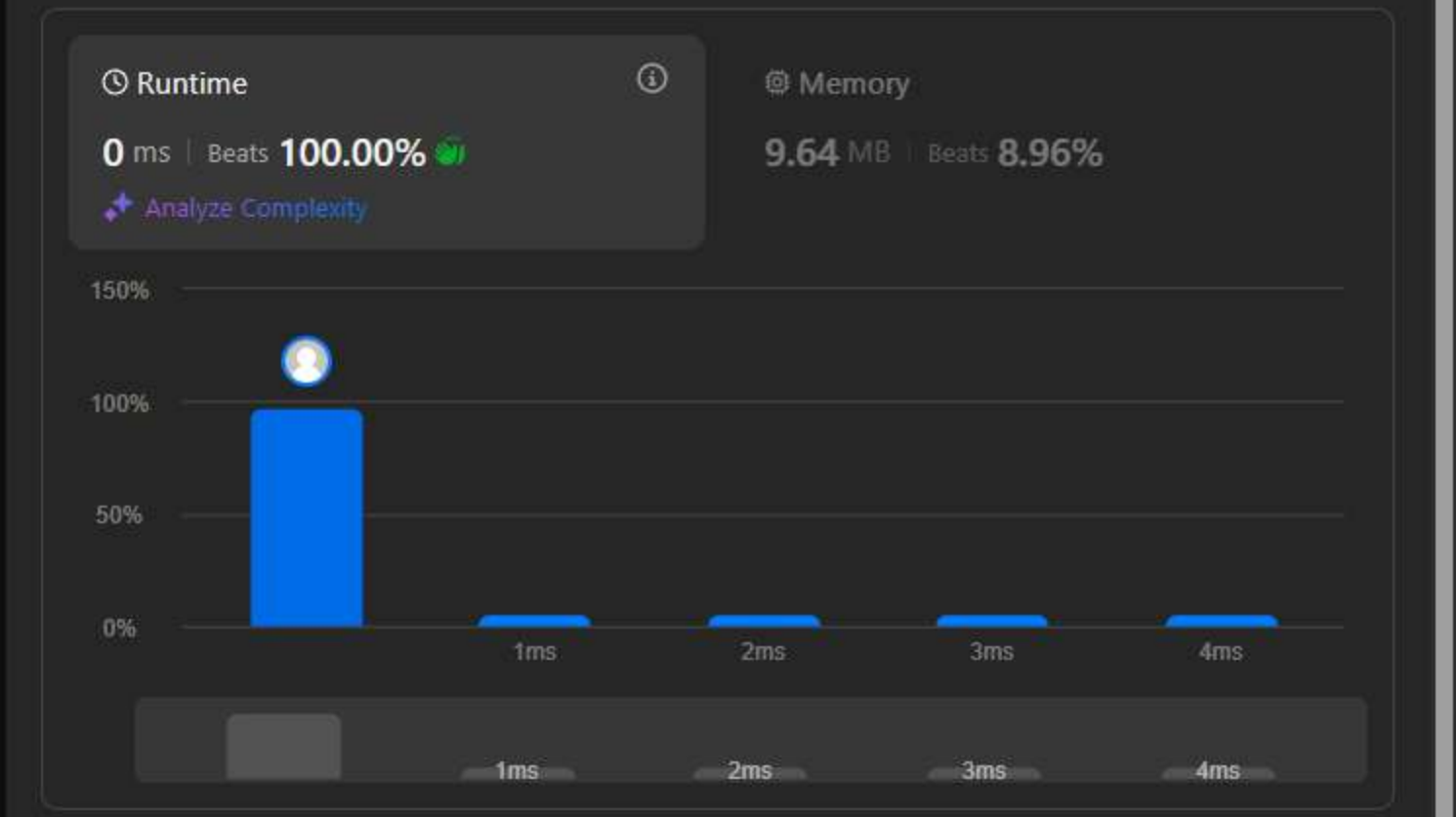
Test Result

Accepted20 / 20 testcases passed

yousab21

submitted at Mar 01, 2026 19:40

EditorialSolution



CodeC++

```
1 //the way i made 2 queues represnt a stack is
2 // at any time there will be an active queue and an empty queue
3 //the active queue front will always match the stack top (ill explain h
4 //so when popping or returning top the active queue works excacly as a s
5
6 #include<queue>
7 class MyStack {
8 public:
```

View more

More challenges

232. Implement Queue using Stacks

Write your notes here

Code

C++Auto

```
31         q1.push(q2.front());
32         q2.pop();
33     }
34     active = 1;
35 }
36 }
37
38 int pop() {
39     int stackTop;
40     if(active == 1){
41         stackTop = q1.front();
42         q1.pop();
43     }
44     else if(active == 2){
45         stackTop = q2.front();
46         q2.pop();
47     }
48     return stackTop;
49 }
50
51 int top() {
52     int stackTop;
53     if(active == 1){
54         stackTop = q1.front();
55     }
56     else if(active == 2){
57         stackTop = q2.front();
58     }
59     return stackTop;
60 }
61
62 bool empty() {
63     return q1.empty() && q2.empty();
64 }
65 };
66
67 /**
68  * Your MyStack object will be instantiated and called as such:
69  * MyStack* obj = new MyStack();
70  * obj->push(x);
71  * int param_2 = obj->pop();
72  * int param_3 = obj->top();
73  * bool param_4 = obj->empty();
74  */
```

Saved

Ln 3, Col 40

DescriptionAccepted x Editorial Solutions Submissions

All Submissions

Accepted 22 / 22 testcases passed

 yousab21 submitted at Mar 01, 2026 20:14

EditorialSolution

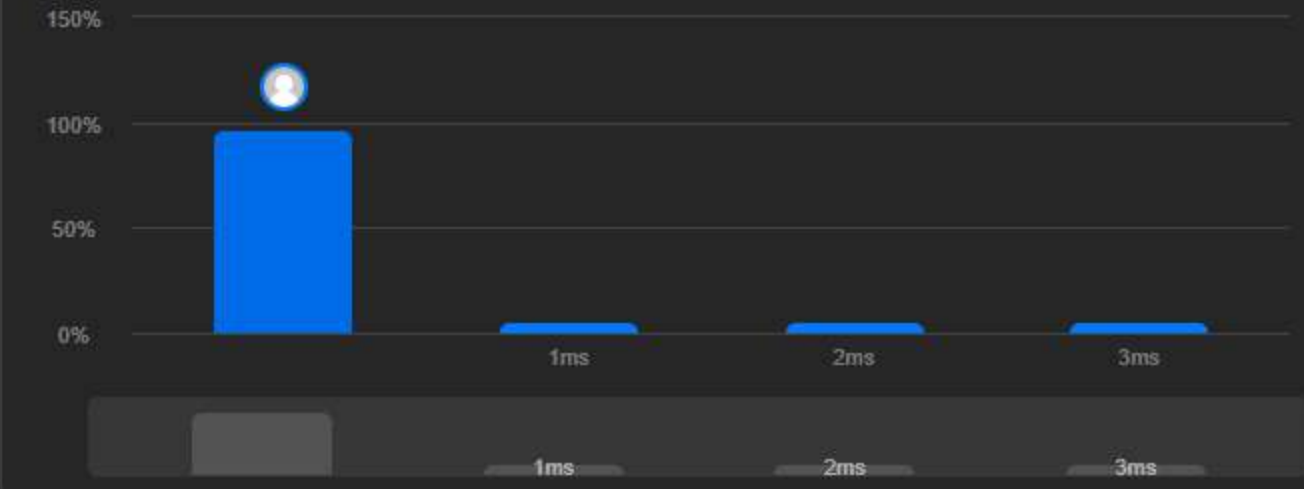
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

9.81 MB | Beats 30.51%



Code | C++

```
1 //this problem is very simmler to the last one except we dont switch active
2 //we just use a helper stack (i will also explain why in the push function)
3 #include<stack>
4 class MyQueue {
5     stack<int> active;
6     stack<int> helper;
7 public:
8     MyQueue() {
```

View more

More challenges

426. Convert Binary Search Tree to Sorted Doubly Linked List

1653. Minimum Deletions to Make String Balanced

2327. Number of People Aware of a Secret

Write your notes here

Code

C++ Auto

```
17 //this problem is very simmler to the last one except we dont switch active stacks
16 //we just use a helper stack (i will also explain why in the push function)
15 #include<stack>
14 class MyQueue {
13     stack<int> active;
12     stack<int> helper;
11 public:
10     MyQueue() {
9     }
8
7     void push(int x) {
6         //to add an element to the queue and access it through a stack we
5         //need to add elements such that queue front is always = the stack top
4         //so we add elements by following this algorithm :
3         //1- reverse all elements into helper stacks so that now the queue back is at the helper top
2         //2- add element to the helper top (so it is added after the queue back)
1         //3- reverse elements back into active stack so that now the element we added is in the bottom of the stack
18         //and the queue front is at the top
1         while(!active.empty()){
2             helper.push(active.top());
3             active.pop();
4         }
5         helper.push(x);
6         while(!helper.empty()){
7             active.push(helper.top());
8             helper.pop();
9         }
10     }
11
12     int pop() {
13         int queueFront = active.top();
14         active.pop();
15         return queueFront;
16     }
17
18     int peek() {
19         int queueFront = active.top();
20         return queueFront;
21     }
22
23     bool empty() {
24         return active.empty();
25     }
26 };
```

Saved

Ln 18, Col 45

Testcase Test Result

Accepted Runtime: 0 ms

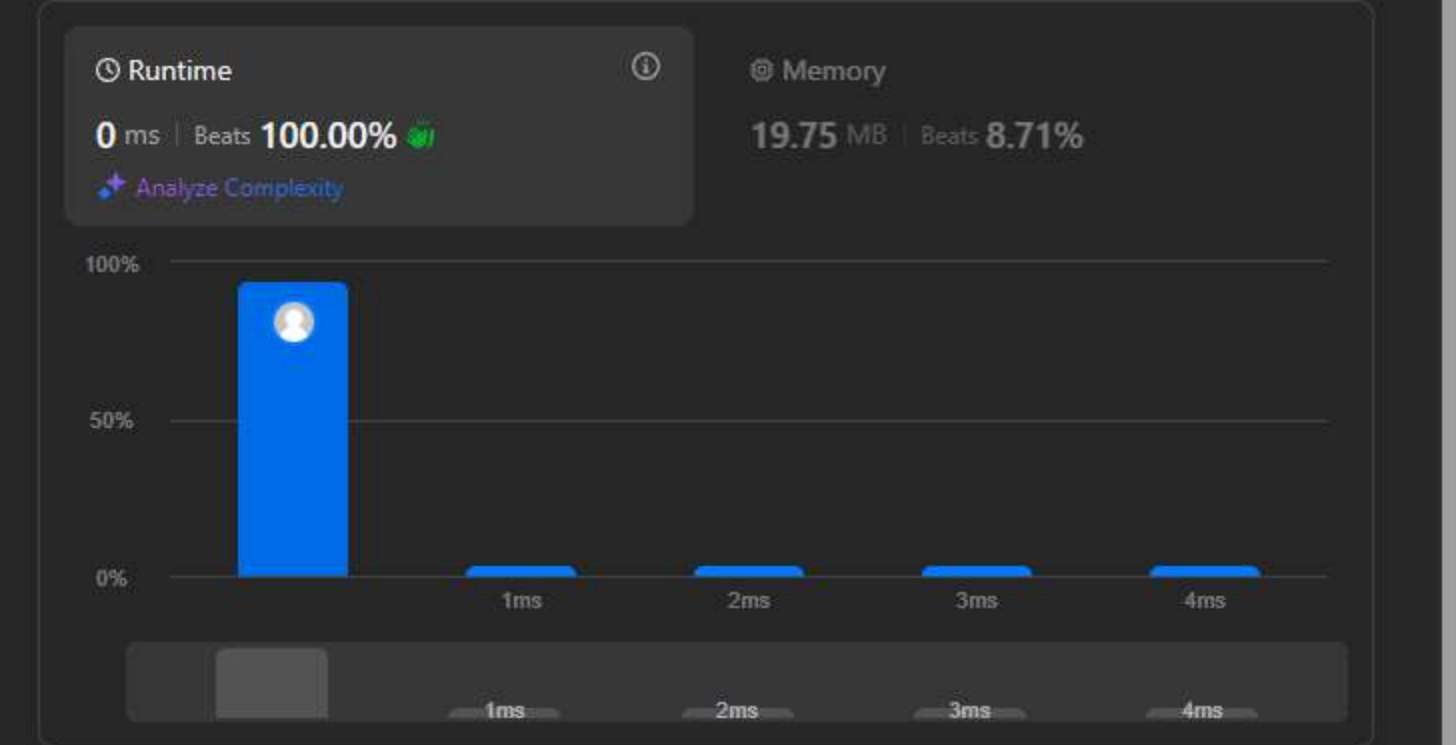
Case 1

Inout

Accepted 208 / 208 testcases passed

yousab21 submitted at Mar 01, 2026 21:03

Editorial Solution



Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
```

View more

- More challenges
88. Merge Sorted Array

244. Shortest Word Distance II

1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags 0/5

```
19
18 class Solution {
17 private:
16     ListNode* resultHead = nullptr;
15     ListNode* resultTail = nullptr;
14
13     void addToList(int x){
12         if(resultHead == nullptr){
11             resultHead = new ListNode(x, nullptr);
10             resultTail = resultHead;
9         }
8         else{
7             ListNode* newNode = new ListNode(x, nullptr);
6             resultTail->next = newNode;
5             resultTail = newNode;
4         }
3     }
2
1 public:
31 //the way i merge those lists is by following this algorithm:
1 //1-start both lists from the head
2 //2-compare the current value in list1 with list2
3 //3-add the smaller one to the result list and go to the next node in this list
4 //4-repeat step 2 and 3 until one of the lists are empty
5 //5-copy the remaining elements from the non empty list
6     ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
7         ListNode* listPtr1 = list1;
8         ListNode* listPtr2 = list2;
9         while(listPtr1 != nullptr && listPtr2 != nullptr){
10             if(listPtr1->val <= listPtr2->val){
11                 addToList(listPtr1->val);
12                 listPtr1 = listPtr1 -> next;
13             }
14             else if(listPtr2->val <= listPtr1->val){
15                 addToList(listPtr2->val);
16                 listPtr2 = listPtr2 -> next;
17             }
18         }
19         while(listPtr1 != nullptr){
20             addToList(listPtr1->val);
21             listPtr1 = listPtr1 -> next;
22         }
23         while(listPtr2 != nullptr){
24             addToList(listPtr2->val);
25             listPtr2 = listPtr2 -> next;
26         }
27         return resultHead;
28     }
29 };
```


Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted

168 / 168 testcases passed

Editorial

Solution

yousab21

submitted at Mar 02, 2026 16:41

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.00 MB | Beats 90.13%



Runtime	Beats
0 ms	100.00%
1 ms	~0.00%
2 ms	~0.00%
3 ms	~0.00%
4 ms	~0.00%

Code

C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10
11 //this is how i solve this:
12 //1- if the list is empty just return the empty list
13 //2- if it is not empty i loop to the element next to last
14 //3- i check if the value in the next element = the value of the current elmenet
15 //4- if true then it is a duplicate i delete it and check again for more duplicates
16 //5- if the next element is not a duplicate i just go to the next iteration
17
18 public:
19     ListNode* deleteDuplicates(ListNode* head) {
20         if (head == nullptr){
21             return nullptr;
22         }
23
24         ListNode* current = head;
25         while(current->next != nullptr){
26             if (current->next->val == current->val ){
27                 ListNode* temp = current->next;
28                 current ->next = temp->next;
29                 delete temp;
30                 temp = nullptr;
31             }
32             else{
33                 current = current->next;
34             }
35         }
36         return head;
37     }
38 };
```

View more

More challenges

82. Remove Duplicates from Sorted List II

1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags

0/5

Code

C++

Auto

```
18 /**
19  * Definition for singly-linked list.
20  * struct ListNode {
21  *     int val;
22  *     ListNode *next;
23  *     ListNode() : val(0), next(nullptr) {}
24  *     ListNode(int x) : val(x), next(nullptr) {}
25  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
26  * };
27
28 class Solution {
29 private:
30
31 //this is how i solve this:
32 //1- if the list is empty just return the empty list
33 //2- if it is not empty i loop to the element next to last
34 //3- i check if the value in the next element = the value of the current elmenet
35 //4- if true then it is a duplicate i delete it and check again for more duplicates
36 //5- if the next element is not a duplicate i just go to the next iteration
37
38 public:
39     ListNode* deleteDuplicates(ListNode* head) {
40         if (head == nullptr){
41             return nullptr;
42         }
43
44         ListNode* current = head;
45         while(current->next != nullptr){
46             if (current->next->val == current->val ){
47                 ListNode* temp = current->next;
48                 current ->next = temp->next;
49                 delete temp;
50                 temp = nullptr;
51             }
52             else{
53                 current = current->next;
54             }
55         }
56         return head;
57     }
58 };
```

Saved

Ln 19, Col 51

Testcase

Test Result

Accepted

Runtime: 0 ms

Description

Accepted

Editorial

Solutions

Submissions

← All Submissions

🔗

Accepted

166 / 166 testcases passed

Editorial

Solution

👤 yousab21

submitted at Mar 02, 2026 17:49

🕒 Runtime

📄 3 ms | Beats 7.96%

📄 Analyze Complexity

💾 Memory

📄 15.90 MB | Beats 14.17%

Code

C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* deleteDuplicates(ListNode* head) {
14         //if the list is empty or have one node then it cant be a duplicate just return it
15         if (head == nullptr || head->next == nullptr){
16             return head;
17         }
18
19         //this piece of code tracks if the next node is a duplicate
20         //if it is not then just go the next node and continue the loop
21         //and we track the node we were on as the "last none duplicate node"
22         //also there is one edge case we need to account for that is if all nodes are duplicates
23         //then we delete all nodes so ptr becomes nullptr
24         //and then if the loop was likw this " while(ptr->next !=nullptr)"
25         //it gives a runtime error because ptr->next is now nullptr->next
26         ListNode *ptr = head;
27         ListNode *prev = nullptr;
28         while( ptr!= nullptr && ptr->next !=nullptr){
29             if( ptr->next->val != ptr->val){
30                 prev = ptr;
31                 ptr = ptr->next;
32                 continue;
33             }
34
35             //if it is a duplicate then we have to track 3 things before we start deleting elements
36             //1- the node just before the duplicate chain (we already did that in the last piece of code)
37             //2- the first node in the deuplicate chain i called it "dupStart"
38             //3- the node just after the duplicate chain (we get it in the next piece of code)
39             else{
40                 ListNode* dupStart = ptr;
41
42                 //this code moves "ptr" to the first node after the loop chain
43                 //so now we have our "node after duplicates" reference
44                 //so we make the node before the dup chain point to the one after it
45                 //and we also handle the edge case that if the head was a part of the dup chain
46                 //andwe have to account for the same edge case as the first loop here too
47                 while( ptr->next != nullptr && ptr->val == ptr->next->val){
48                     ptr = ptr->next;
49                 }
50                 if (prev != nullptr)
51                     prev->next = ptr;
52                 else
53                     head = ptr;
54                 ptr = ptr->next;
55             }
56         }
57         return head;
58     }
59 }
```

View more

More challenges

• 1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags

0/5

Code

C++

Auto

```
74 /**
73  * Definition for singly-linked list.
72  * struct ListNode {
71  *     int val;
70  *     ListNode *next;
69  *     ListNode() : val(0), next(nullptr) {}
68  *     ListNode(int x) : val(x), next(nullptr) {}
67  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
66  * };
65  */
64
63 //this one is a bit more confusing ill explain my solution as i go in the code
62 class Solution {
61
60 public:
59     ListNode* deleteDuplicates(ListNode* head) {
58         //if the list is empty or have one node then it cant be a duplicate just return it
57         if (head == nullptr || head->next == nullptr){
56             return head;
55         }
54
53         //this piece of code tracks if the next node is a duplicate
52         //if it is not then just go the next node and continue the loop
51         //and we track the node we were on as the "last none duplicate node"
50         //also there is one edge case we need to account for that is if all nodes are duplicates
49         //then we delete all nodes so ptr becomes nullptr
48         //and then if the loop was likw this " while(ptr->next !=nullptr)"
47         //it gives a runtime error because ptr->next is now nullptr->next
46         ListNode *ptr = head;
45         ListNode *prev = nullptr;
44         while( ptr!= nullptr && ptr->next !=nullptr){
43             if( ptr->next->val != ptr->val){
42                 prev = ptr;
41                 ptr = ptr->next;
40                 continue;
39             }
38
37             //if it is a duplicate then we have to track 3 things before we start deleting elements
36             //1- the node just before the duplicate chain (we already did that in the last piece of code)
35             //2- the first node in the deuplicate chain i called it "dupStart"
34             //3- the node just after the duplicate chain (we get it in the next piece of code)
33             else{
32                 ListNode* dupStart = ptr;
31
30                 //this code moves "ptr" to the first node after the loop chain
29                 //so now we have our "node after duplicates" reference
28                 //so we make the node before the dup chain point to the one after it
27                 //and we also handle the edge case that if the head was a part of the dup chain
26                 //andwe have to account for the same edge case as the first loop here too
25                 while( ptr->next != nullptr && ptr->val == ptr->next->val){
48                     ptr = ptr->next;
49                 }
50                 if (prev != nullptr)
51                     prev->next = ptr;
52                 else
53                     head = ptr;
54                 ptr = ptr->next;
55             }
56         }
57         return head;
58     }
59 }
```

Saved

Ln 75, Col 37

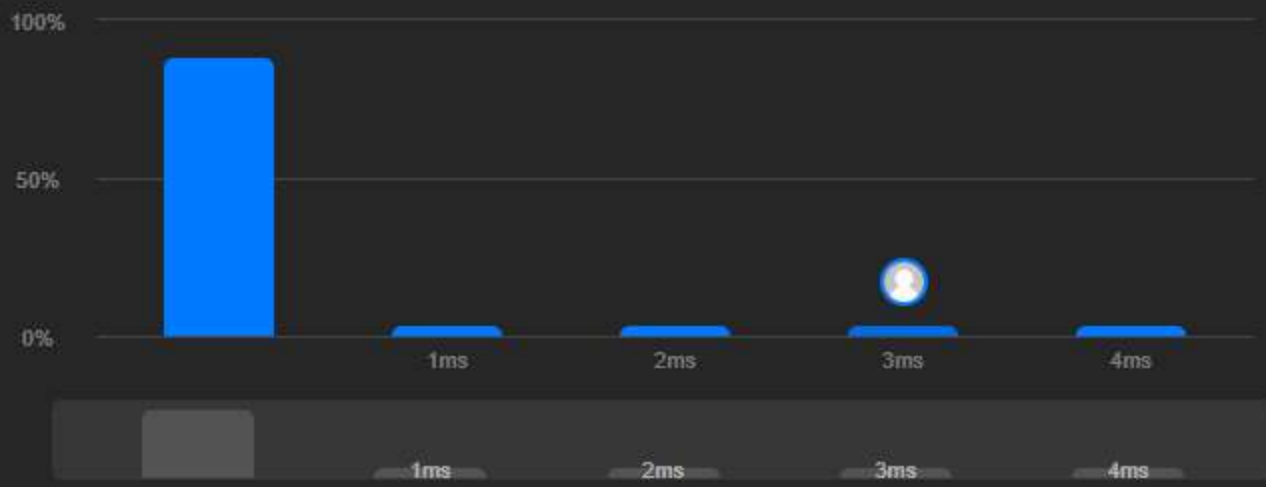
Testcase

Test Result

Accepted 166 / 166 testcases passed
yousab21 submitted at Mar 02, 2026 17:49

Editorial Solution

Runtime 3 ms | Beats 7.96%
Memory 15.90 MB | Beats 14.17%



Code C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
```

More challenges

1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
47 //it gives a runtime error because ptr->next is now nullptr->next
46 ListNode *ptr = head;
45 ListNode *prev = nullptr;
44 while( ptr!= nullptr && ptr->next !=nullptr){
43     if( ptr->next->val != ptr->val){
42         prev = ptr;
41         ptr = ptr->next;
40         continue;
39     }
38
37 //if it is a duplicate then we have to track 3 things before we start deleting elements
36 //1- the node just before the duplicate chain (we already did that in the last piece of code)
35 //2- the first node in the deuplicate chain i called it "dupStart"
34 //3- the node just after the duplicate chain (we get it in the next piece of code)
33 else{
32     ListNode* dupStart = ptr;
31
30     //this code moves "ptr" to the first node after the loop chain
29     //so now we have our "node after duplicates" reference
28     //so we make the node before the dup chain point to the one after it
27     //and we also handle the edge case that if the head was a part of the dup chain
26     //andwe have to account for the same edge case as the first loop here too
25     while( ptr->next != nullptr && ptr->val == ptr->next->val){
24         ptr = ptr->next;
23     }
22     ptr = ptr->next;
21     if(head == dupStart){
20         head = ptr;
19     }
18     else{
17         prev->next = ptr;
16     }
15
14     //this code just deletes all nodes from the first node in the dup chain to the
13     //first node after the dup chain
12     ListNode *temp = dupStart;
11     while (dupStart != ptr){
10         temp = dupStart;
9         dupStart = dupStart->next;
8         delete temp;
7     }
6 }
5 }
4 return head;
3 }
2 //leet code shows that this problem can be solved in 0ms runtime
1 //i cant find that solution ill wait for what the instructor has
75 //to say about it in the session
1 };
```

Saved

Ln 75, Col 37

Testcase Test Result

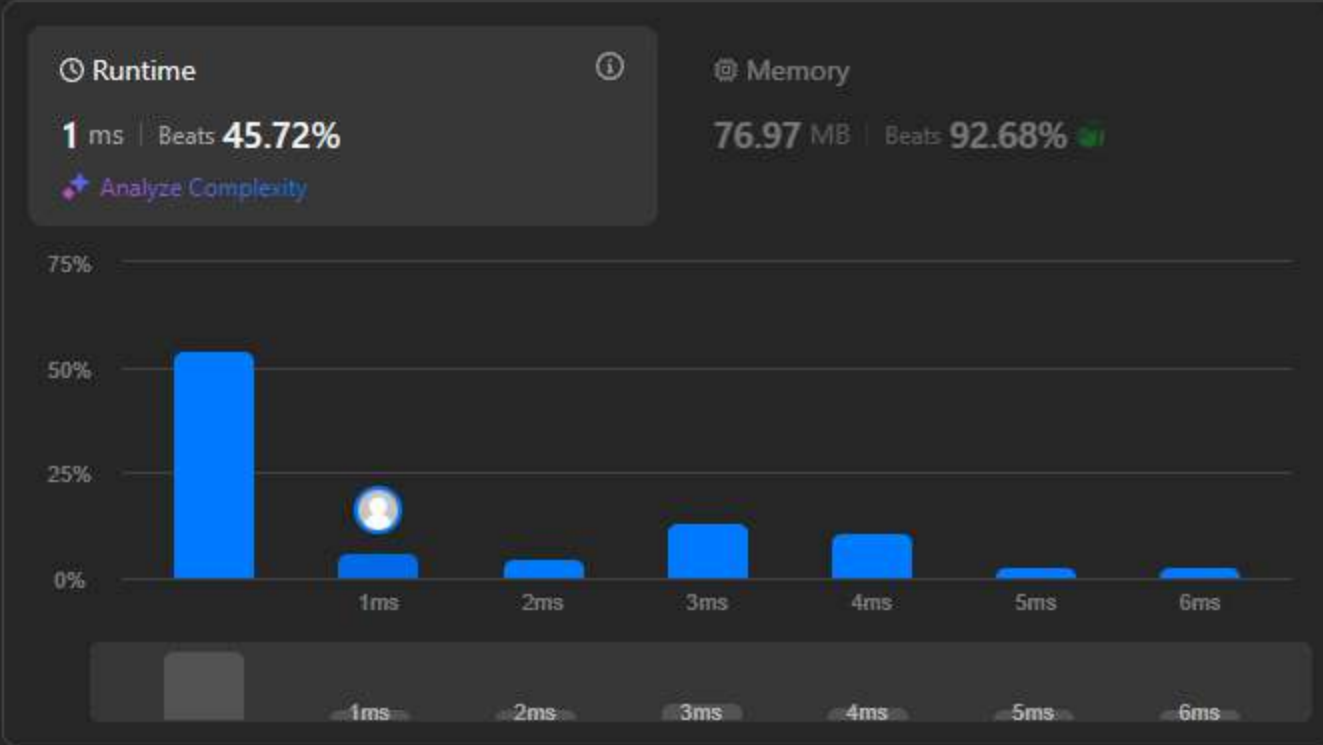
Accepted 1569 / 1569 testcases passed

👤 yousab21

 submitted at Mar 02, 2026 18:58

Editorial

Solution



Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
```

View more

- More challenges
43. Multiply Strings

445. Add Two Numbers II

1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags

0/5

```
68 //this problem is very similar to another one we had at week1
69 //same idea with diffrence implementation to represent a linked list
70 //here is my solution:
71
72 class Solution {
73     ListNode* resultHead = nullptr;
74     ListNode* resultTail = nullptr;
75
76     //this is just a helper function to add elements to the anser list
77     void addToResult(int val){
78         ListNode* newNode = new ListNode(val, nullptr);
79         if(resultHead == nullptr){
80             resultHead = newNode;
81             resultTail = newNode;
82         }
83         else{
84             resultTail->next = newNode;
85             resultTail = newNode;
86         }
87     }
88
89 public:
90     ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
91         //first i declare a digit and a carry values to use in the addition operation
92         int digit = 0, carry = 0;
93
94         //i calculate the value of the next digit by getting the sum of the matching values in lists + tha carry if it exists
95         //i then add said value in the result array dirictly if it dont exceed 9
96         //it does it gets split into a digit that gets added and a carry to be summed up with the next digit
97         //i repeat this until one of the lists runs out of elements
98         while(l1 != nullptr && l2 != nullptr){
99             digit = l1->val + l2->val + carry;
100             carry = 0;
101             if(digit > 9){
102                 carry = digit / 10;
103                 digit = digit % 10;
104             }
105             addToResult(digit);
106             l1 = l1->next;
107             l2 = l2->next;
108         }
109
110         //if one list is empty and the other stil has elements (one had more digits than the other)
111         //i copy the remaining elements to the result while still being mindfull of the carry
112         while(l1 != nullptr) {
113             digit = l1->val + carry;
114             carry = 0;
115             if(digit > 9){
116                 carry = digit / 10;
117                 digit = digit % 10;
118             }
119             addToResult(digit);
120             l1 = l1->next;
121         }
122         return resultHead;
123     }
124 }
```


Accepted 1569 / 1569 testcases passed

yousab21 submitted at Mar 02, 2026 18:58

Editorial

Solution

Runtime

1 ms | Beats 45.72%

Analyze Complexity

Memory

76.97 MB | Beats 92.68%



Code C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

More challenges

43. Multiply Strings 445. Add Two Numbers II

1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
39 //it does it gets split into a digit that gets added and a carry to be summed up with the next digit
38 //i repeat this until one of the lists runs out of elements
37 while(l1 != nullptr && l2 != nullptr){
36     digit = l1->val + l2->val + carry;
35     carry = 0;
34     if(digit > 9){
33         carry = digit / 10;
32         digit = digit % 10;
31     }
30     addToResult(digit);
29     l1 = l1->next;
28     l2 = l2->next;
27 }
26
25 //if one list is empty and the other still has elements (one had more digits than the other)
24 //i copy the remaining elements to the result while still being mindful of the carry
23 while(l1 != nullptr) {
22     digit = l1->val + carry;
21     carry = 0;
20     if(digit > 9){
19         carry = digit / 10;
18         digit = digit % 10;
17     }
16     addToResult(digit);
15     l1 = l1->next;
14 }
13 while(l2 != nullptr) {
12     digit = l2->val + carry;
11     carry = 0;
10     if(digit > 9){
9         carry = digit / 10;
8         digit = digit % 10;
7     }
6     addToResult(digit);
5     l2 = l2->next;
4 }
3
2 //this if condition handles an edge case that the last digit to be pushed was itself greater than 9
1 //in that case following the code above we should have already added up the digit
80 //so we add the remaining carry into a new node
1 if(carry > 0){
2     addToResult(carry);
3 }
4 return resultHead;
5
6 };
```

Saved

Ln 80, Col 56

Testcase Test Result

 **Solution**

55.726mb 58.264mb 60.801mb 63.339mb 65.876mb 68.414mb 70.951mb 73.489mb

View more

0/5

Testcase > Test Result

Linked List

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted30 / 30 testcases passed

youstab21 submitted at Mar 03, 2026 19:02

Runtime

2421 ms | Beats 5.15%

Analyze Complexity

Memory

55.94 MB | Beats 96.03%

Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

More challenges

75. Sort Colors

147. Insertion Sort List

2046. Sort Linked List Already Sorted Using Absolute Values

Write your notes here

Select related tags0/5

Code

C++ vAuto

```
19
18 //this algorithm works by walking a variable commonly called "key" over every element of the list and
17 //compares every element before it to it
16 //find the correct place for the current key
15 //this creates an area at the start that with be sorted within itself
14 //the elements in this area might not be in thier final position but they are sorted within themself
13 ListNode* keyPrev = head;
12 ListNode* key = head->next;
11
10 while(key != nullptr) {
9     //if the element before the key is smaller than it then we are is said sorted area
8     //so we just walk the key until we get to the first unsorted element
7     if(key->val >= keyPrev->val){
6         keyPrev = key;
5         key = key->next;
4     }
3     else{
2
1     //now we need to find where the current key should fit in the sorted area we have
52 //so we walk a scanner on it until if finds a value greter of equal than key (we should add the key right before this value)
1     ListNode *scanner = preHead;
2     while(scanner->next != key && scanner->next->val < key->val){
3         scanner = scanner->next;
4     }
5     //scanner now is poining to the element the key should be after
6
7     //as this area is sorted and we always track the element before the key
8     //we can simply insert the key into its ordered position and link the rest of the list
9     //using the variabe used to track the element before the key
10    keyPrev ->next = key->next;
11    key->next = scanner->next;
12    scanner -> next = key;
13
14    //now that the key node is in then correct posision we can continue walking the key from we left off
15    key = keyPrev->next;
16
17 }
18
19 //as the first element can by at the wrong place we cant just return the head of the list
20 //as we update the prehead->next pointer when we add an element after it
21 ListNode *newHead = preHead->next;
22 delete preHead; //now we just delete the prehead beacuse we no longer need it
23 return newHead;
24
25
26 };
```

Saved

Ln 52, Col 115

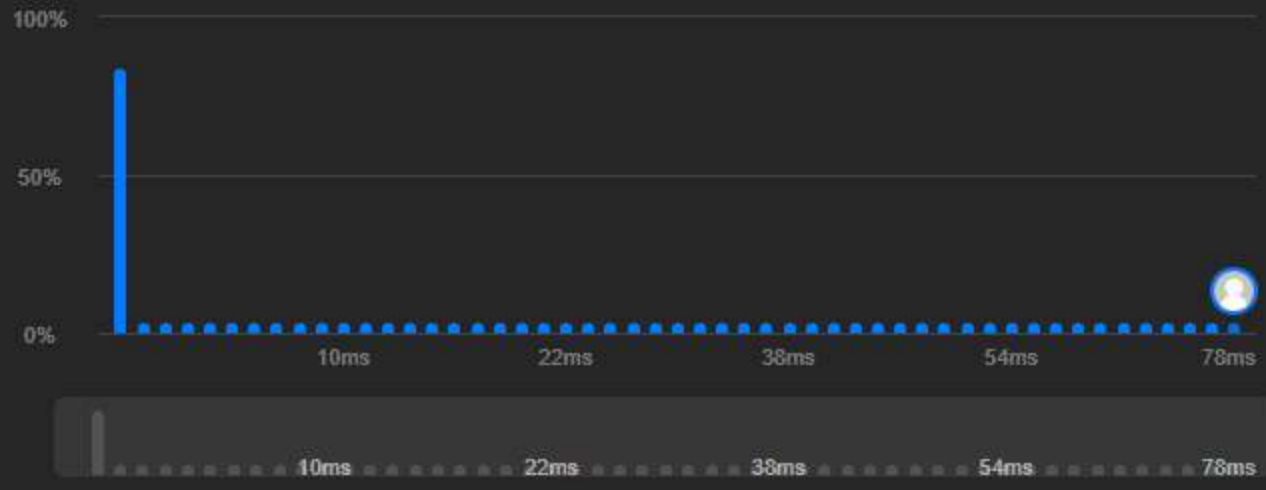
TestcaseTest Result

Accepted 12 / 12 testcases passed
yousab21 submitted at Mar 03, 2026 19:59

Editorial Solution

Runtime
1388 ms | Beats **5.02%**
Analyze Complexity

Memory
22.84 MB | Beats **54.54%**



Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 struct ListNode {
12     int val;
13     ListNode *next;
14     ListNode() : val(0), next(nullptr) {}
15     ListNode(int x) : val(x), next(nullptr) {}
16     ListNode(int x, ListNode *next) : val(x), next(next) {}
17 };
18
19 class Solution {
20 public:
21     void reorderList(ListNode* head) {
22         // the reason i want to get the element before the tail not the tail itself
23         // is because the problem wants the tail to be moved so we need to call the function again
24         // just to make the newTail->next = nullptr so that i made it return the element before the head
25         // to store it in a variable to try to save a function call
26         ListNode* getBeforeTail(ListNode* head) {
27             ListNode* temp = head;
28             while(temp->next->next != nullptr) {
29                 temp = temp->next;
30             }
31             return temp;
32         }
33         void reorderList(ListNode* head) {
34             if(head == nullptr || head->next == nullptr) {
35                 return;
36             }
37             ListNode *current = head;
38             ListNode *next = current->next;
39
40             // here i loop over the elements and add the current tail in between
41             // current and next and update the new current to be equal to next
42             // which moves it 2 spots in the new list landing at the next element
43             // i want to repeat the logic after
44             while(current->next != nullptr && current->next->next != nullptr) {
45                 next = current->next;
46                 ListNode* beforeTail = getBeforeTail(head);
47                 ListNode* tail = beforeTail->next;
48                 current->next = tail;
49                 tail->next = next;
50                 current = next;
51                 beforeTail->next = nullptr;
52             }
53         }
54     };
55 };
```

More challenges

- 2095. Delete the Middle Node of a Linked List
- 2516. Take K of Each Character From Left and Right

Write your notes here

Select related tags 0/5

Code Submit Ctrl Enter

C++ Auto

```
30 *     ListNode(int x) : val(x), next(nullptr) {}
29 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
28 * };
27 */
26
25
24 class Solution {
23 public:
22     // the reason i want to get the element before the tail not the tail itself
21     // is because the problem wants the tail to be moved so we need to call the function again
20     // just to make the newTail->next = nullptr so that i made it return the element before the head
19     // to store it in a variable to try to save a function call
18     ListNode* getBeforeTail(ListNode* head) {
17         ListNode* temp = head;
16         while(temp->next->next != nullptr) {
15             temp = temp->next;
14         }
13         return temp;
12     }
11 void reorderList(ListNode* head) {
10     if(head == nullptr || head->next == nullptr) {
9         return;
8     }
7
6     ListNode *current = head;
5     ListNode *next = current->next;
4
3     // here i loop over the elements and add the current tail in between
2     // current and next and update the new current to be equal to next
1     // which moves it 2 spots in the new list landing at the next element
37 // i want to repeat the logic after
1     while(current->next != nullptr && current->next->next != nullptr) {
2         next = current->next;
3         ListNode* beforeTail = getBeforeTail(head);
4         ListNode* tail = beforeTail->next;
5         current->next = tail;
6         tail->next = next;
7         current = next;
8         beforeTail->next = nullptr;
9     }
10 }
11 };
```

Saved Ln 37, Col 43

Testcase Test Result

Accepted Runtime: 0 ms

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted28 / 28 testcases passed

yousab21 submitted at Mar 03, 2026 20:30

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

13.40 MB | Beats 72.27%

Runtime	Beats
0 ms	100.00%
1 ms	~0%
2 ms	~0%
3 ms	~0%
4 ms	~0%

Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10
11 //this is just the stander algorithm in the material
12
13 class Solution {
14 public:
15     ListNode* reverseList(ListNode* head) {
16         if(head == nullptr || head->next == nullptr){
17             return head;
18         }
19         //assing the 3 pointer and walk them next to one another
20         //reversing pointer i the porcces until the neode before last
21         ListNode* prev = nullptr;
22         ListNode* current = head;
23         ListNode* next = head->next;
24
25         while(next != nullptr){
26             current->next = prev;
27             prev = current;
28             current = next;
29             next=next->next;
30         }
31         //reverse the last pointer and return the head
32         current->next = prev;
33         head = current;
34         return head;
35     };
36 };
```

View more

More challenges

156. Binary Tree Upside Down

234. Palindrome Linked List

2807. Insert Greatest Common Divisors in Linked List

Write your notes here

Select related tags

Code

C++ Auto

```
30 /**
31  * Definition for singly-linked list.
32  * struct ListNode {
33  *     int val;
34  *     ListNode *next;
35  *     ListNode() : val(0), next(nullptr) {}
36  *     ListNode(int x) : val(x), next(nullptr) {}
37  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
38  * };
39
40 //this is just the stander algorithm in the material
41
42 class Solution {
43 public:
44     ListNode* reverseList(ListNode* head) {
45         if(head == nullptr || head->next == nullptr){
46             return head;
47         }
48         //assing the 3 pointer and walk them next to one another
49         //reversing pointer i the porcces until the neode before last
50         ListNode* prev = nullptr;
51         ListNode* current = head;
52         ListNode* next = head->next;
53
54         while(next != nullptr){
55             current->next = prev;
56             prev = current;
57             current = next;
58             next=next->next;
59         }
60         //reverse the last pointer and return the head
61         current->next = prev;
62         head = current;
63         return head;
64     };
65 };
```

Saved

Ln 31, Col 55

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

head =

Linked List

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted 44 / 44 testcases passed

yousab21 submitted at Mar 04, 2026 19:38

EditorialSolution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

11.34 MB | Beats 38.21%

Code C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

More challenges

109. Convert Sorted List to Binary Search Tree

1019. Next Greater Node In Linked List

2816. Double a Number Represented as a Linked List

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
40 class Solution {
39
38     //i made this function to be able to get a pointer relative to an
37     //index in the linked list this will make this alot simpler
36     //node than the index starts from 1 in this problem
35     ListNode* getPointerToPosition(ListNode* head, int i){
34         ListNode* current = head;
33         while(i > 1 && current != nullptr){
32             current = current->next;
31             i--;
30         }
29         return current;
28     }
27
26 public:
25     ListNode* reverseBetween(ListNode* head, int left, int right) {
24         //usual edge case if the list is empty or has 1 node
23         if(head == nullptr || head->next == nullptr){
22             return head;
21         }
20         //here i make 3 pointer to point to
19         //beforeRev : the last node befoer the sublist i want to reverse
18         //startRev : the first node in the sublist i want to revrse
17         //afterrev : the fisrt node after the sublist i want to reverse
16         //i will use these pointer to glue back the list after reversing tghe sublist
15         ListNode* beforeRev = getPointerToPosition(head, left - 1);
14         ListNode* startRev = getPointerToPosition(head, left);
13         ListNode* afterRev = getPointerToPosition(head, right + 1);
12         //this is an edge case for if the beging of the sublist was the first elemtn
11         //in that case we need a preHead pointer to act as a refrence as in this case
10         //the head node will be moved
9         if(left == 1) {
8             beforeRev = new ListNode(0, head);
7             startRev = head;
6         }
5
4         //these are the normal 3 pointers used to reversed the list identical to the last problem
3         //except the current node starts as the start of the sublist i want to reverse
2         ListNode* prev = nullptr;
1         ListNode* current = startRev;
51         ListNode* next = current->next;
1
2         //i start revesing the sublist starting from the start of the sublist
3         //until the end of it or until the list ends (edge case if right was the end of the list)
4         while(current != afterRev && current != nullptr){
5             next = current->next;
6             current->next = prev;
7             prev = current;
8             current = next;
9         }
52     }
53 }
```

Saved

Ln 51, Col 40

TestcaseTest Result

Linked List

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted 44 / 44 testcases passed

yousab21 submitted at Mar 04, 2026 19:38

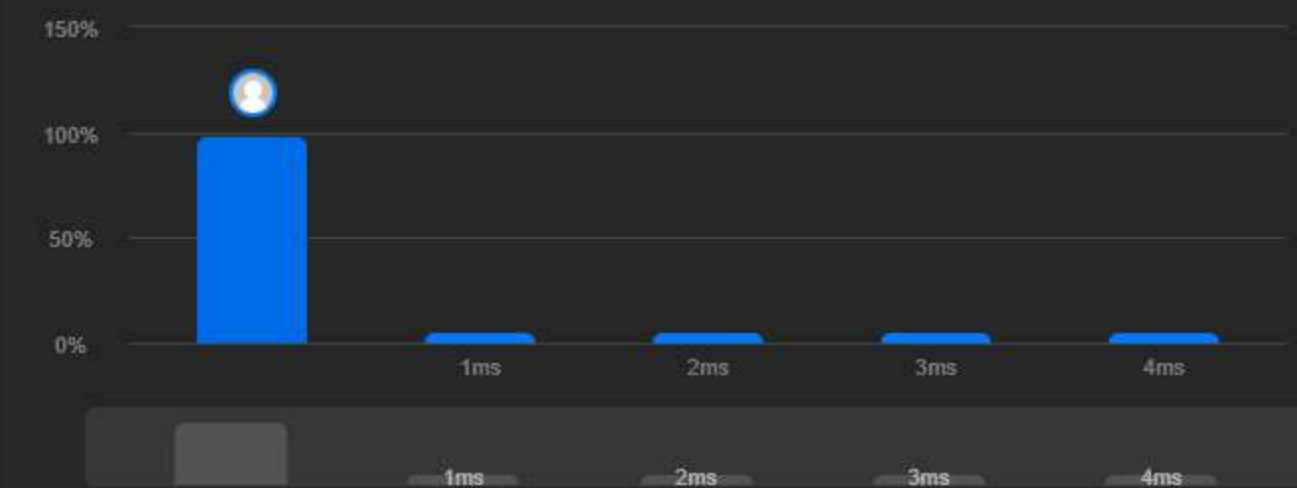
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

11.34 MB | Beats 38.21%



Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

Write your notes here

Select related tags 0/5

Code

C++ v Auto

```
20 //here i make 3 pointer to point to
19 //beforeRev : the last node befoer the sublist i want to reverse
18 //startRev : the first node in the sublist i want to revrse
17 //afterrev : the first node after the sublist i want to reverse
16 //i will use these pointer to glue back the list after reversing tghe sublist
15 ListNode* beforeRev = getPointerToPosition(head, left - 1);
14 ListNode* startRev = getPointerToPosition(head, left);
13 ListNode* afterRev = getPointerToPosition(head, right + 1);
12 //this is an edge case for if the beging of the sublist was the first elemtn
11 //in that case we need a preHead pointer to act as a refrence as in this case
10 //the head node will be moved
9 if(left == 1) {
8     beforeRev = new ListNode(0,head);
7     startRev = head;
6 }
5
4 //these are the normal 3 pointers used to reversed the list identical to the last problem
3 //except the current node starts as the start of the sublist i want to reverse
2 ListNode* prev = nullptr;
1 ListNode* current = startRev;
51 ListNode* next = current->next;
1
2 //i start revesing the sublist starting from the start of the sublist
3 //until the end of it or until the list ends (edge case if right was the end of the list)
4 while(current != afterRev && current!=nullptr){
5     next= current->next;
6     current->next = prev;
7     prev = current;
8     current = next;
9 }
10
11 //here is glue back the reversed sublist to the rest of the list
12 //node that after revesring "prev" becomes the head of the new revered sublist
13 // and "startRev" becomes the tail of it
14 beforeRev->next = prev;
15 startRev->next =afterRev;
16
17 //this condition is for the the same edge case where we start reversing from the head
18 //in this case the head of the new list is the head of the revered sublist
19 if(left == 1){
20     head = prev;
21 }
22
23 return head;
24
25 };
```

Saved

Ln 51, Col 40

Testcase Test Result

Linked List

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted232 / 232 testcases passed

youstab21submitted at Mar 04, 2026 20:23

EditorialSolution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.36 MB | Beats 64.06%

Runtime	1ms	2ms	3ms	4ms
0 ms	100%	~1%	~1%	~1%

CodeC++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

More challenges

189. Rotate Array

725. Split Linked List in Parts

Write your notes here

Select related tags0/5

Submit

Ln 46, Col 73

Code

C++Auto

```
35 class Solution {
36     //i reused the same getBeforeTail function i built for the reorder problem
37     ListNode* getBeforeTail(ListNode* head){
38         ListNode* temp = head;
39         while(temp->next->next != nullptr){
40             temp = temp->next;
41         }
42         return temp;
43     }
44
45     //this a function that gets the leanth of the linked list
46     //this is very important for a step ill explain later
47     int getLength(ListNode* head){
48         ListNode* current = head;
49         int length = 0;
50         while (current != nullptr){
51             current = current->next;
52             length++;
53         }
54         return length;
55     }
56
57 public:
58     ListNode* rotateRight(ListNode* head, int k) {
59         //usual edge case
60         if (head==nullptr || head->next ==nullptr || k == 0){
61             return head;
62         }
63
64         //=====//
65         int n = getLength(head);
66         k = k % n;
67         //=====//
68
69         //in my opinion this is the main reason the problem is rated mediam not easy
70         //by the problem constraintns k could be so big that leetcode wont except the anser
71         //so we solve this by taking advantage of the fact that rotating the list n times
72         //just ends up the same as the original list so we need to eliminate multipls of n
73         //from k until it is smaller than n at firstst i was coding this as a loop until i realized
74         //the "%" operator does excacly that
75
76         //inside this loop is the logic to make the last node the new
77         //head i just repeat it k times
78         for(int i = 0; i < k ; i++){
79             ListNode* beforeTail = getBeforeTail(head);
80             ListNode* tail = beforeTail->next;
81
82             //simple logic to make the tail the new head
83             //just make the tail->next point to head
84             //make the tail te new haed
85             //detach the tail from the before tail node
86         }
87     }
88 }
```

Saved

TestcaseTest Result

Accepted 232 / 232 testcases passed

yousab21 submitted at Mar 04, 2026 20:23

Editorial

Solution

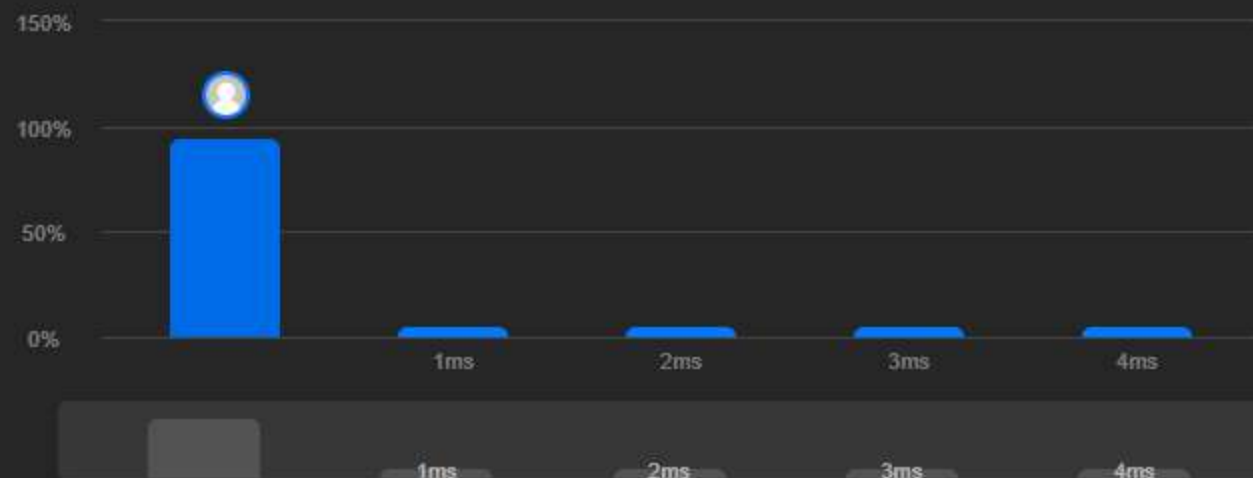
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.36 MB | Beats 64.06%



Code C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * }
```

View more

More challenges

189. Rotate Array

725. Split Linked List in Parts

Write your notes here

Select related tags

0/5

Code

Submit Ctrl Enter

C++ Auto

```
24 //this is very important for a step ill explain later
23 int getLength(ListNode* head){
22     ListNode* current = head;
21     int length = 0;
20     while (current != nullptr){
19         current = current->next;
18         length++;
17     }
16     return length;
15 }
14
13 public:
12     ListNode* rotateRight(ListNode* head, int k) {
11         //usual edge case
10         if (head==nullptr || head->next ==nullptr || k == 0){
9             return head;
8         }
7
6         //=====//
5         int n = getLength(head);
4         k = k % n;
3         //=====//
2         //in my opinion this is the main reason the problem is rated mediam not easy
1         //by the problem constraintns k could be so big that leetcode wont except the anser
46 //so we solve this by taking advantage of the fact that rotating the list n times
1         //just ends up the same as the original list so we need to eliminate multipls of n
2         //from k until it is smaller than n at firtst i was coding this as a loop until i realized
3         //the "%" operator does excacly that
4
5         //inside this loop is the logic to make the last node the new
6         //head i just repeat it k times
7         for(int i = 0; i < k ; i++){
8             ListNode* beforeTail = getBeforeTail(head);
9             ListNode* tail = beforeTail->next;
10
11             //simple logic to make the tail the new head
12             //just make the tail->next point to head
13             //make the tail te new haed
14             //detach the tail from the before tail node
15             tail->next = head;
16             head=tail;
17             beforeTail->next= nullptr;
18         }
19         return head;
20     }
21 };
```

Saved

Ln 46, Col 73

Testcase Test Result

Linked List

DescriptionAccepted xEditorialSolutionsSubmissions

All Submissions

Accepted41 / 41 testcases passed

youstab21 submitted at Mar 04, 2026 20:37

Runtime

3 ms | Beats 95.31%

Analyze Complexity

Memory

12.36 MB | Beats 55.69%

CodeC++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode(int x) : val(x), next(NULL) {}
7  * };
8  */
```

View more

More challenges

203. Remove Linked List Elements

2487. Remove Nodes From Linked List

3217. Delete Nodes From Linked List Present in Array

Write your notes here

Select related tags0/5

Code

C++Auto

```
16 /**
15  * Definition for singly-linked list.
14  * struct ListNode {
13  *     int val;
12  *     ListNode *next;
11  *     ListNode(int x) : val(x), next(NULL) {}
10  * };
9  */
8
7 //this idea with this problem is we cant delete a specifice node without
6 //a refrence to its previos node as that will beark the list
5 //so what we do is we copy the vlaue of the next node and store
4 //in our refrence node and then linking it to the node after the node we copied
3 // unlinking the node we copied in the process
2 class Solution {
1 public:
17     void deleteNode(ListNode* node) {
1         node->val = node->next->val;
2         node->next = node->next->next;
3     }
4 };
```

Saved

Ln 17, Col 38

TestcaseTest Result